

VIRTUALIZATION: CPU TO MEMORY

CS 537, Spring 2026

Michael Swift

Fun article: <https://lwn.net/Articles/1051430/>



Would you rather live not knowing something or having answers that might be wrong:

<https://go.wisc.edu/p2fh79>

Richard Feynman quote

I think it's much more interesting to live not knowing than to have answers which might be wrong. I have approximate answers and possible beliefs and different degrees of uncertainty about different things, but I am not absolutely sure of anything and there are many things I don't know anything about, such as whether it means anything to ask why we're here. I don't have to know an answer. I don't feel frightened not knowing things, by being lost in a mysterious universe without any purpose, which is the way it really is as far as I can tell.

Studying for this Class

Attend lecture

Read the book

Try the practice problems in the book

Practice exam questions (to be posted soon)

AGENDA / LEARNING OUTCOMES

CPU virtualization

- Recap of scheduling policies

- Work through problems

Memory virtualization

- What is the need for memory virtualization?

- How to virtualize memory?

RECAP: CPU VIRTUALIZATION

RECAP: SCHEDULING MECHANISM

Process: Abstraction to virtualize CPU

Use **time-sharing** in OS to switch between processes

Limited Direct Execution

- Use system calls to run access devices etc. from user mode

- Context-switch using interrupts for multi-tasking

Scheduling Pop-Quiz

Tick, switch, tick again
Too short and we thrash in pain
Too long feels unfair

Q: Which scheduler is this,
and what tradeoff does
the poem describe when
choosing its key
parameter?

Jobs line up to run
Shortest whispers, “Pick me first”
Mean wait time sinks low

Q: Which scheduling
algorithm is being
described, and *why* does it
minimize average waiting
time?

CPU SUMMARY

Mechanism

- Process abstraction

- System call for protection

- Context switch to time-share

Policy

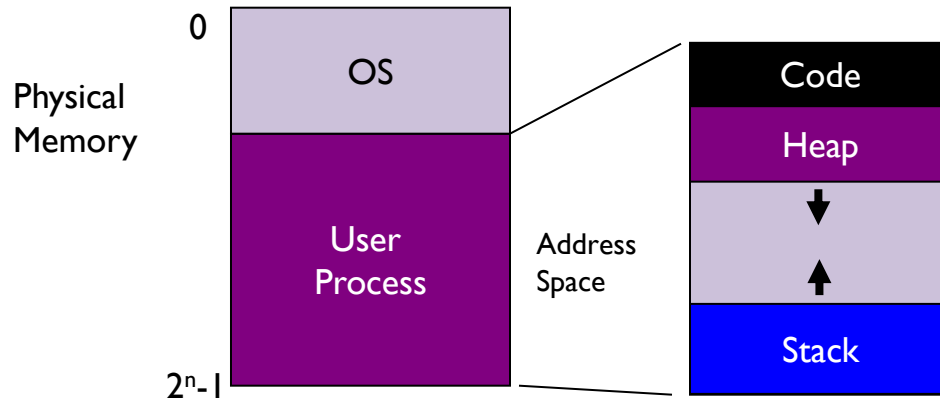
- Metrics: turnaround time, response time

- Balance using MLFQ

VIRTUALIZING MEMORY

Motivation for Virtualization

Uniprogramming: One process runs at a time



Disadvantages:

- Only one process runs at a time
- Process can destroy OS

MULTIPROGRAMMING GOALS

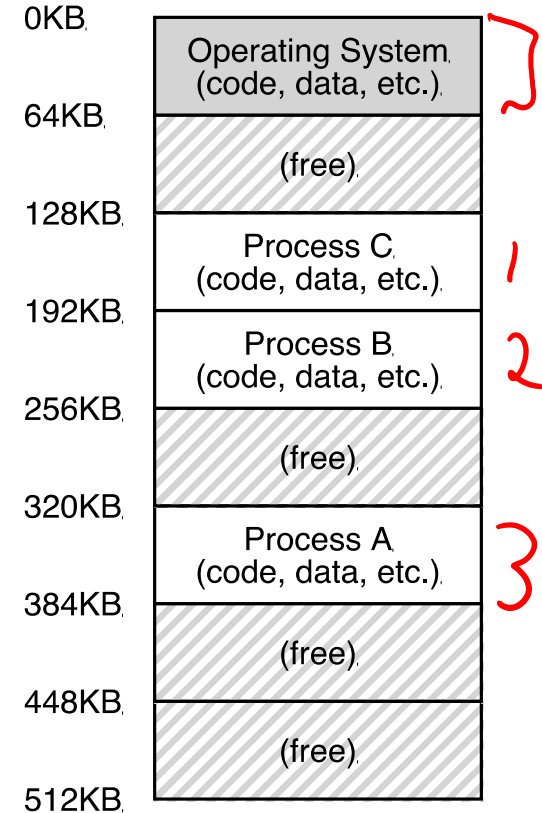
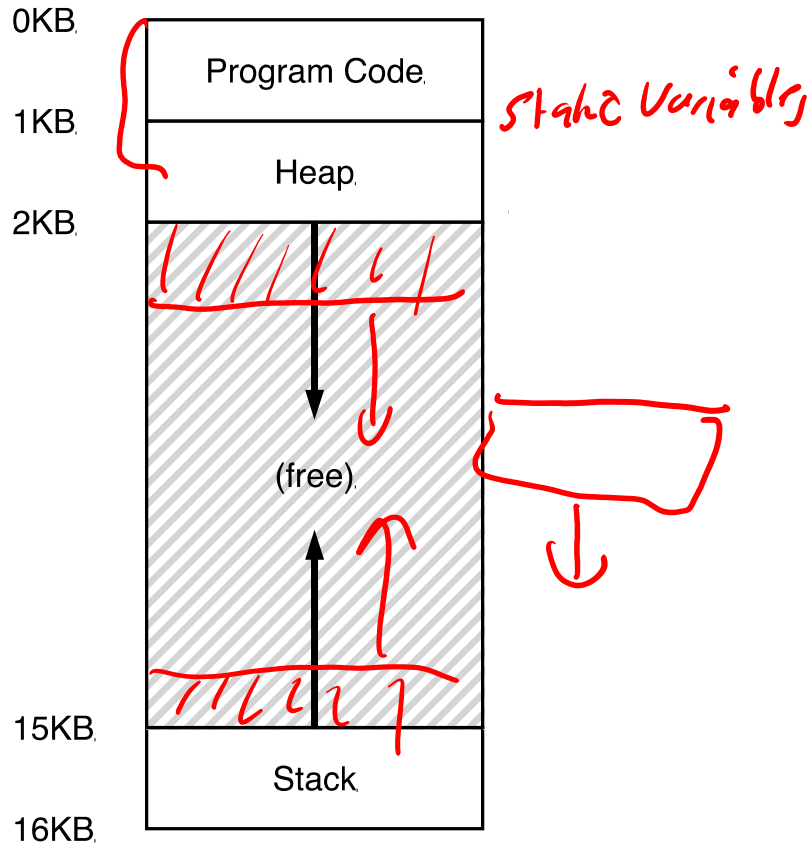
Transparency: Process is unaware of sharing

Protection: Cannot corrupt OS or other process memory

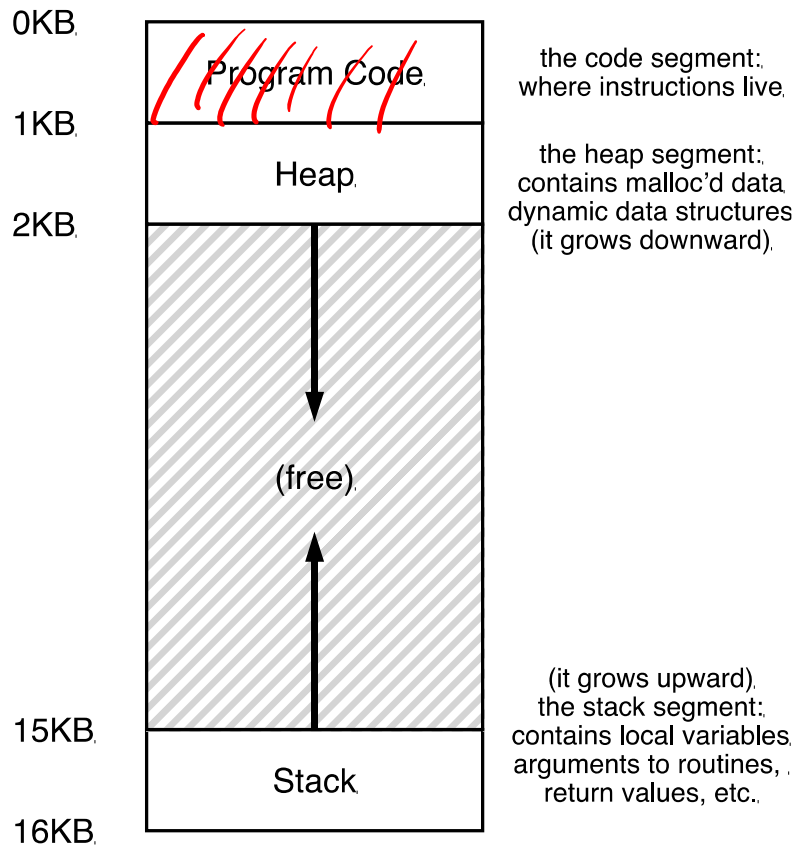
Efficiency: Do not waste memory or slow down processes

Sharing: Enable sharing between cooperating processes

ABSTRACTION: ADDRESS SPACE



WHAT IS IN ADDRESS SPACE?



Static: Code and some global variables

Dynamic: Stack and Heap

STACK ORGANIZATION

```
alloc(A);
```

```
alloc(B);
```

```
alloc(C);
```

```
free(C);
```

```
alloc(D);
```

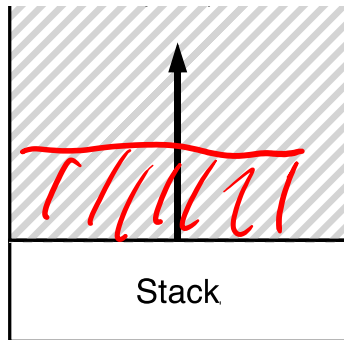
```
free(D);
```

```
free(B);
```

```
free(A);
```

15KB

16KB



Stack grows downward

Pointer between allocated and free space

Allocate: Decrement pointer

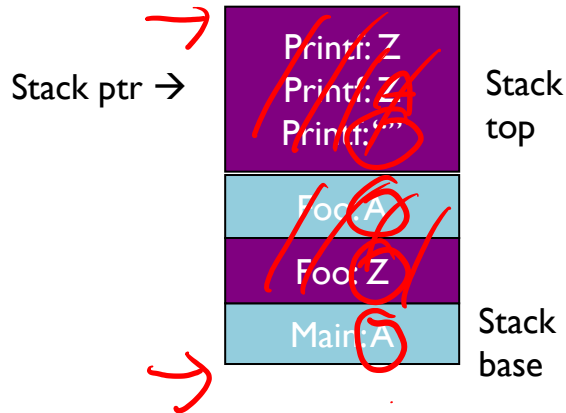
Free: Increment pointer

No fragmentation!

WHAT GOES ON STACK?

```
main () {  
    int A = 0;  
    foo(A);  
    printf("A: %d\n", A);  
}
```

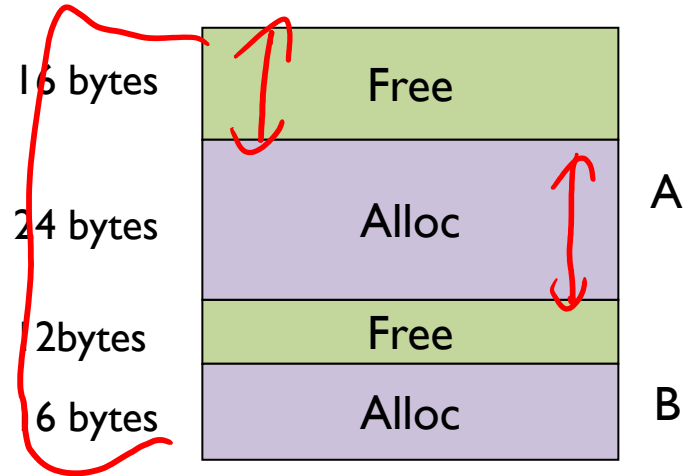
```
void foo (int Z) {  
    int A = 2;  
    Z = 5;  
    printf("A: %d Z: %d\n", A, Z);  
}
```



HEAP ORGANIZATION

Allocate from any random location: malloc(), new() etc.

- Heap memory consists of allocated and free areas (holes)
- Order of allocation and free is unpredictable



Fragmentation
- Free space
not usable

MEMORY ACCESS

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(int argc, char *argv[]) {
    int x;
    x = x + 3;
}
```

```
0x10: movl  -0x8(%rbp), %rdi
0x13: addl  $0x3, %rdi
0x19: movl  %rdi, -0x8(%rbp)
```

%rbp is the base pointer:
points to base of current stack frame

MEMORY ACCESS

Initial %rip = 0x10

%rbp = 0x200

➡ 0x10: movl -0x8(%rbp), %rdi
0x13: addl \$0x3, %rdi
0x19: movl %rdi, -0x8(%rbp)

%**rbp** is the base pointer:
points to base of current stack frame

%**rip** is instruction pointer (or program counter)

MEMORY ACCESS

Initial %rip = 0x10

%rbp = 0x200

➡ 0x10: movl -0x8(%rbp), %rdi
0x13: addl \$0x3, %rdi
0x19: movl %rdi, -0x8(%rbp)

%**rbp** is the base pointer:
points to base of current stack frame

%**rip** is instruction pointer (or program counter)

Fetch instruction at addr 0x10

Exec:

load from addr 0x1f8

Fetch instruction at addr 0x13

Exec:

no memory access

Fetch instruction at addr 0x19

Exec:

store to addr 0x1f8

Memory practice

```
int x;  
int main(int argc, char *argv[]) {  
    int y;  
    int* z = malloc(sizeof(int));  
}
```

Possible locations:
static data/code, stack, heap

Address	Location
x	
main	
y	
z	
*z	

Memory practice Solution

```
int x;  
int main(int argc, char *argv[]) {  
    int y;  
    int* z = malloc(sizeof(int));  
}
```

Possible locations:
static data/code, stack, heap

Address	Location
x	Static data/code
main	Static data/code
y	Stack
z	Stack
*z	Heap

HOW TO VIRTUALIZE MEMORY

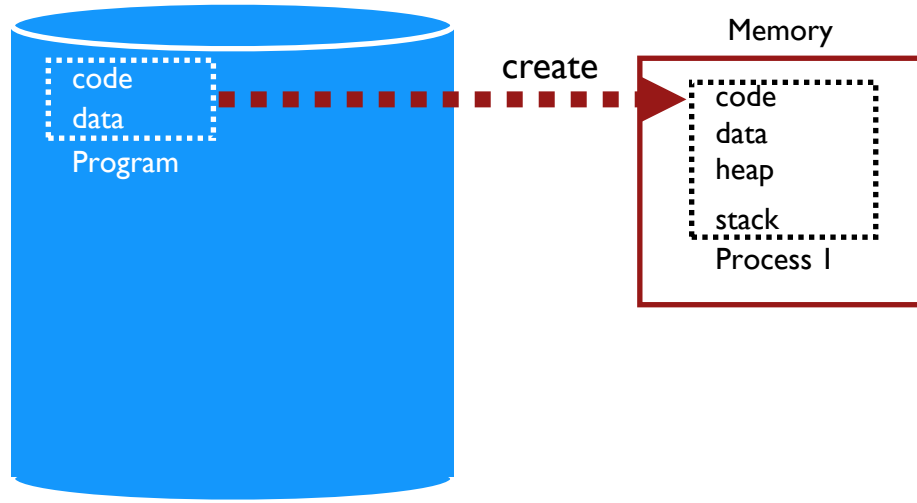
Problem: How to run multiple processes simultaneously?

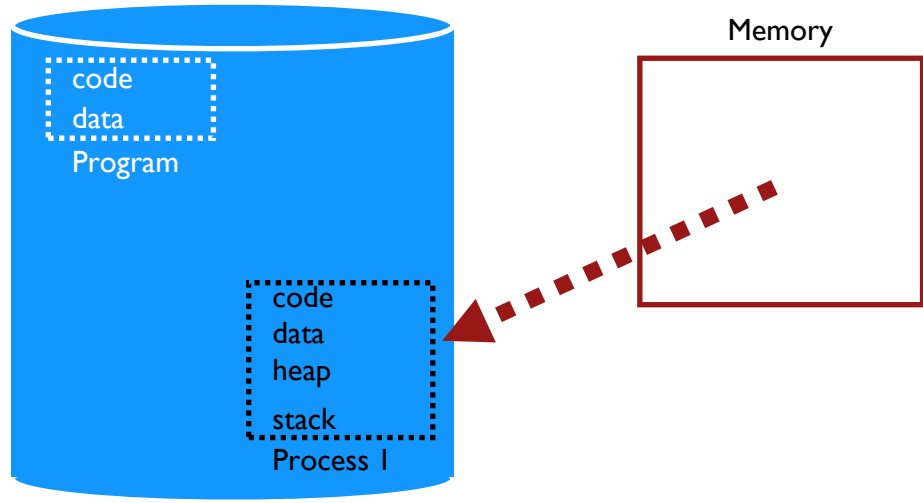
Addresses are “hardcoded” into process binaries

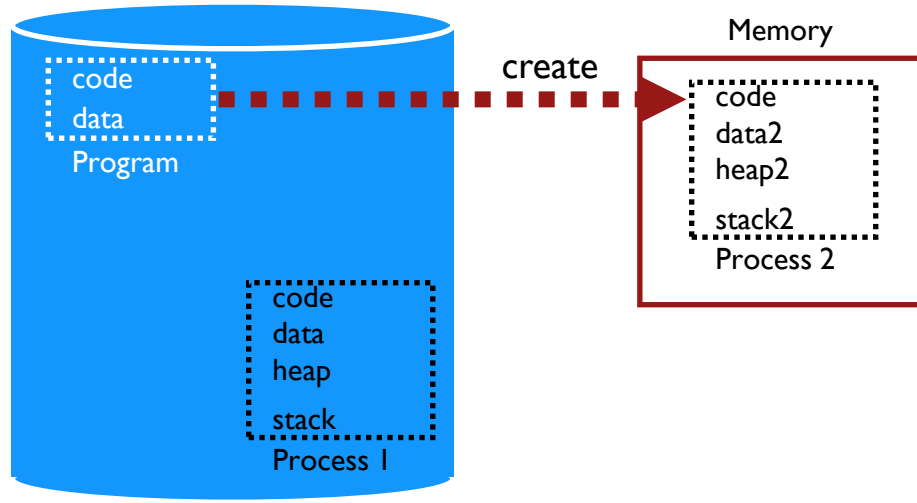
How to avoid collisions?

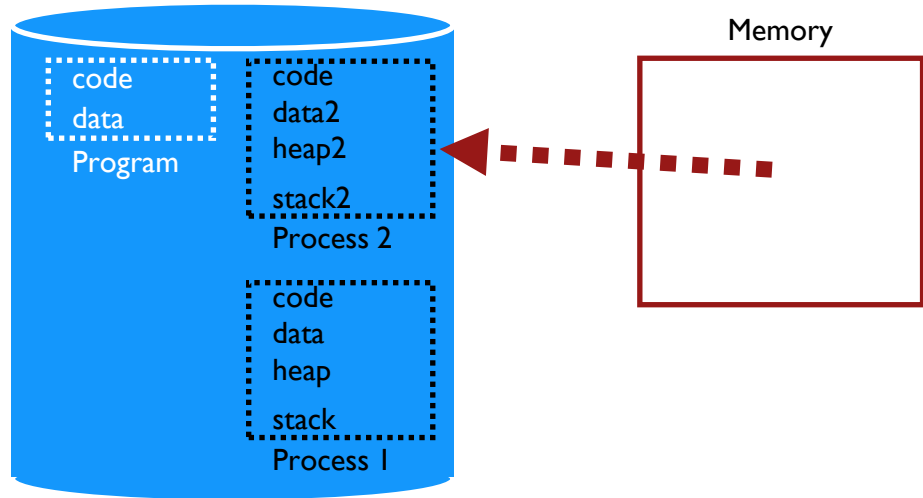
Possible Solutions for Mechanisms (covered today):

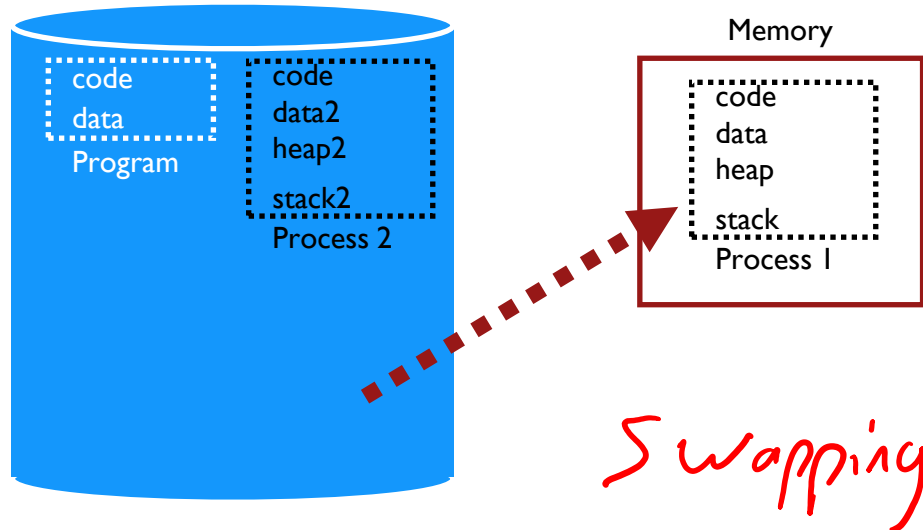
1. Time Sharing
2. Static Relocation
3. Base
4. Base+Bounds











PROBLEMS WITH TIME SHARING?

Ridiculously poor performance

Better Alternative: space sharing!

At same time, space of memory is divided across processes

Remainder of solutions all use space sharing

2) Static Relocation

Idea: OS rewrites each program before loading it as a process in memory

Each rewrite for different process uses different addresses and pointers

Change jumps, loads of static data

```
0x10:movl 0x8(%rbp), %rdi  
0x13:addl $0x3, %rdi  
0x19:movl %rdi, 0x8(%rbp)
```

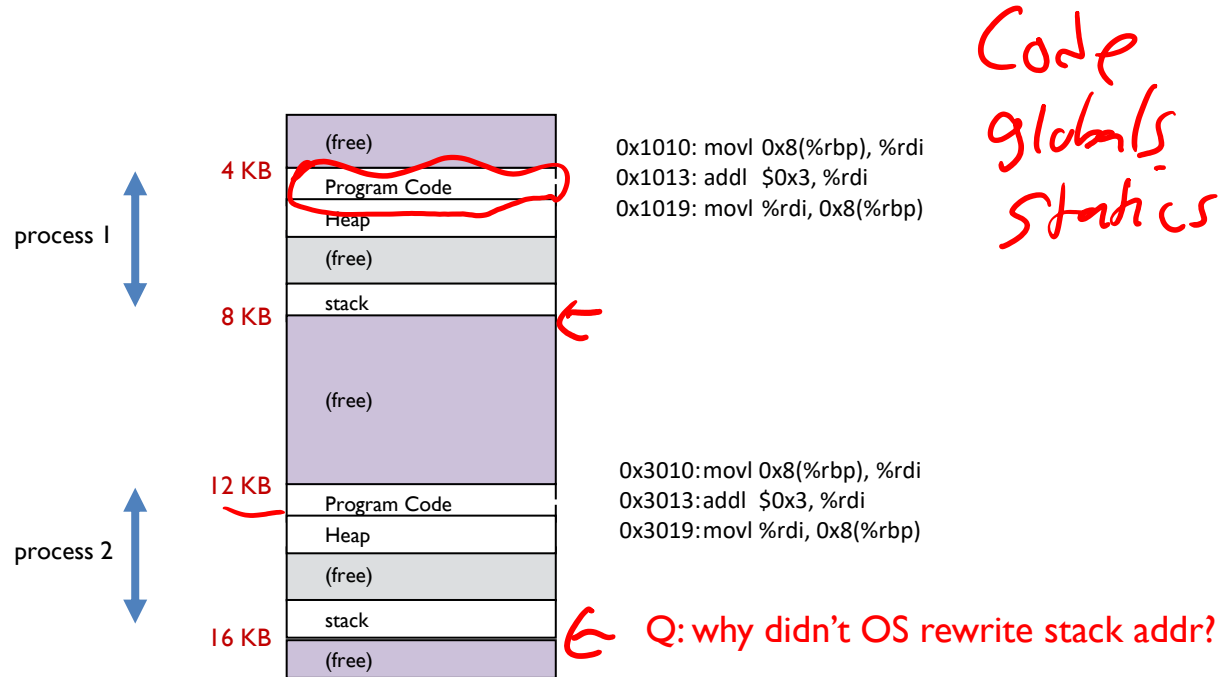
rewrite

```
0x1010:movl 0x8(%rbp), %rdi  
0x1013:addl $0x3, %rdi  
0x1019:movl %rdi, 0x8(%rbp)
```

rewrite

```
0x3010: movl 0x8(%rbp), %rdi  
0x3013: addl $0x3, %rdi  
0x3019: movl %rdi, 0x8(%rbp)
```

Static: Layout in Memory



Static Relocation: Disadvantages

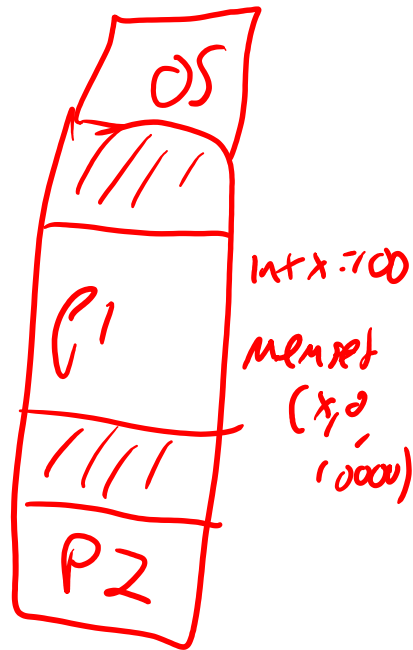
No protection

- Process can destroy OS or other processes
- No privacy



Cannot move address space after it has been placed

- May not be able to allocate new process



3) Dynamic Relocation

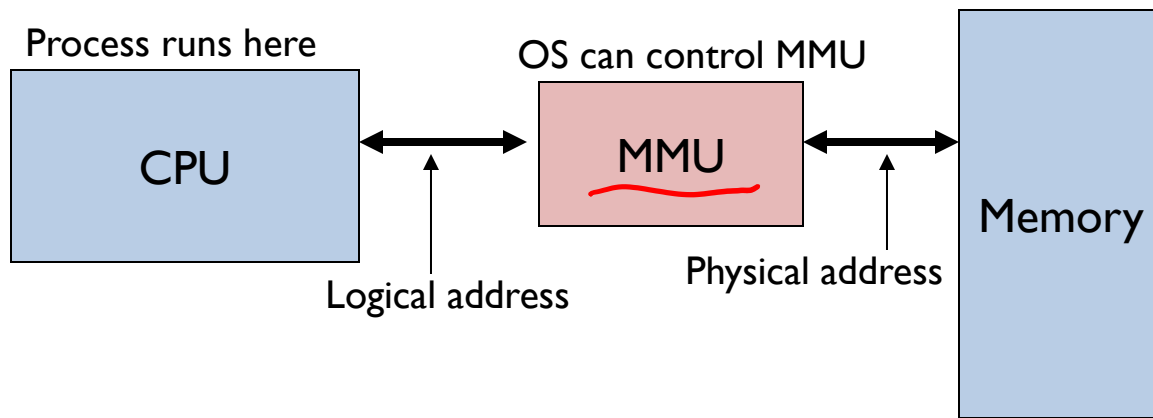
Goal: Protect processes from one another

Requires hardware support

- Memory Management Unit (MMU)

MMU dynamically changes process address at every memory reference

- Process generates **logical** or **virtual** addresses (in their address space)
- Memory hardware uses **physical** or **real** addresses



Virtual Addresses

To make it easier to manage memory of multiple processes, make processes use **virtual addresses**

- virtual addresses are independent of location in physical memory (RAM) that referenced data lives
 - OS determines location in physical memory
- instructions issued by CPU reference virtual addresses
 - e.g., pointers, arguments to load/store instruction, PC, ...
- virtual addresses are translated by hardware into physical addresses (with some help from OS)

Hardware Support for Dynamic Relocation

Privileged (protected, kernel) mode: OS runs

- When enter OS (trap, system calls, interrupts, exceptions)
- Allows certain instructions to be executed
(Can manipulate contents of MMU)
- Allows OS to access all of physical memory

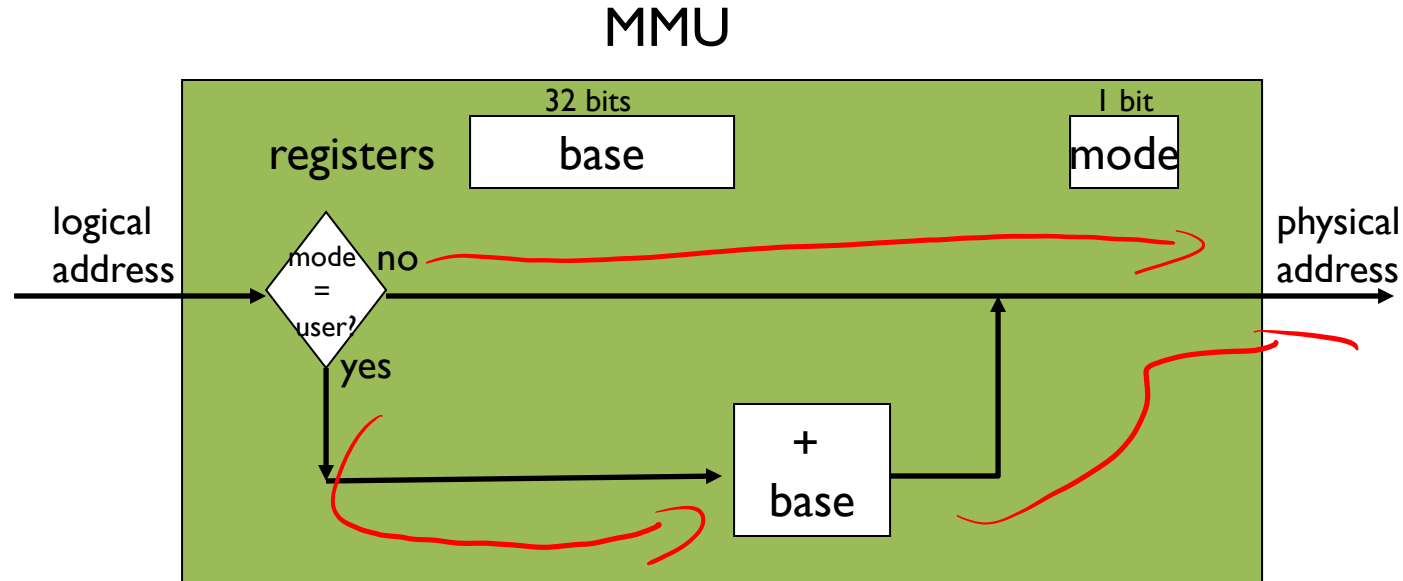
User mode: User processes run

- Perform translation of logical address to physical address

Implementation of Dynamic Relocation: BASE REG

Translation on every memory access of user process

MMU adds base register to logical address to form physical address



Dynamic Relocation with Base Register

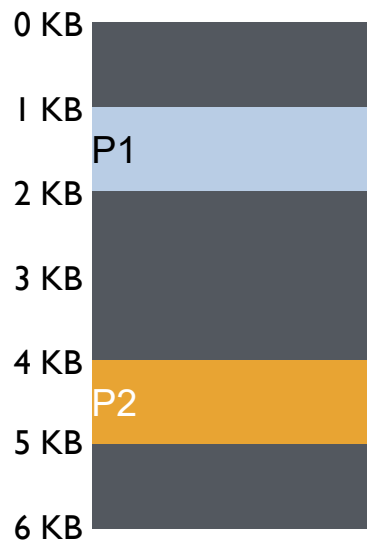
Translate virtual addresses to physical by adding a fixed offset each time.

Store offset in base register

Each process has different value in base register

Dynamic relocation by changing value of base register!

Visual Example of Dynamic Relocation with Base Register



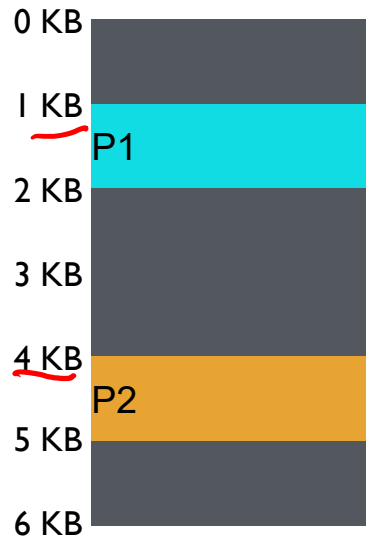
Virtual

P1: load 100, R1

P2: load 100, R1

P2: load 1000, R1

P1: load 100, R1



Virtual

P1: load 100, R1

P2: load 100, R1

P2: load 1000, R1

P1: load 1000, R1

Physical

load 1124, R1

load 4196, R1

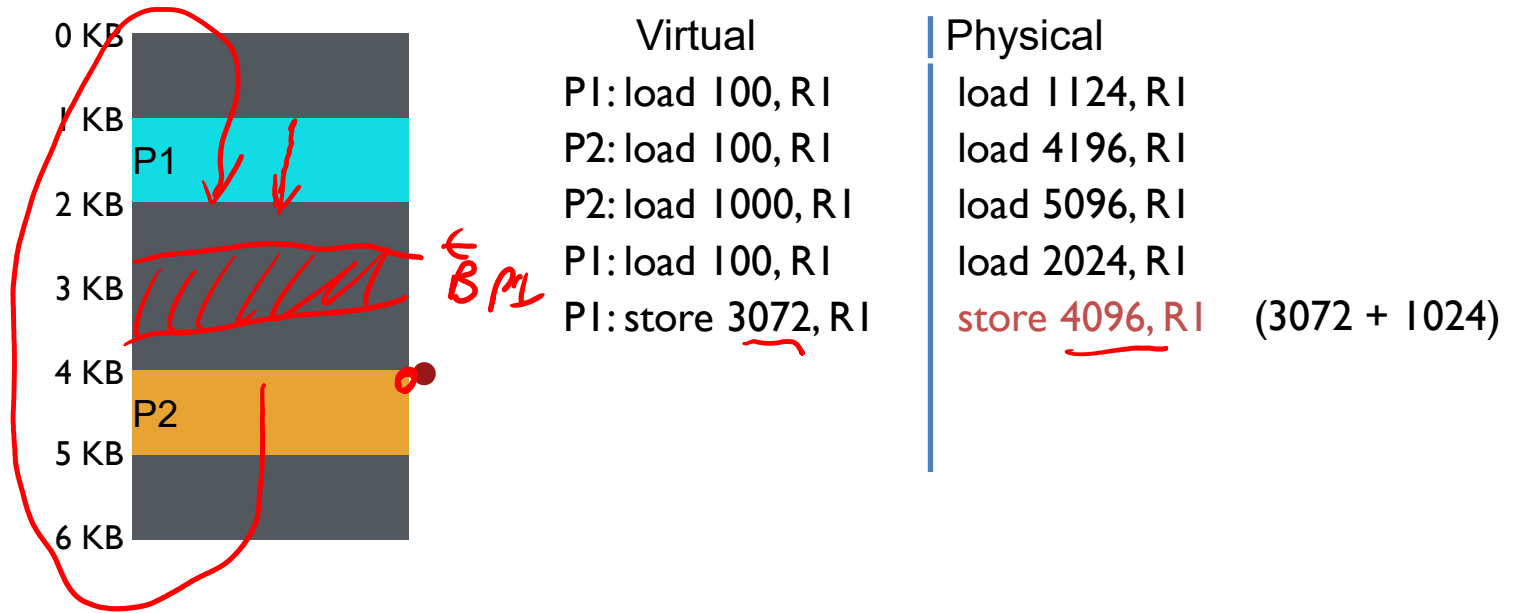
load 5096, R1

load 2024, R1

Can P2 hurt P1?

Can P1 hurt P2?

How well does dynamic relocation do with base register for protection?



How well does dynamic relocation do with base register for protection?

4) Dynamic with Base+Bounds

Idea: limit the address space with a bounds register

Base register: smallest physical addr (or starting location)

Bounds register: size of this process's virtual address space

- Sometimes defined as largest physical address (base + size)

OS kills process if process loads/stores beyond bounds

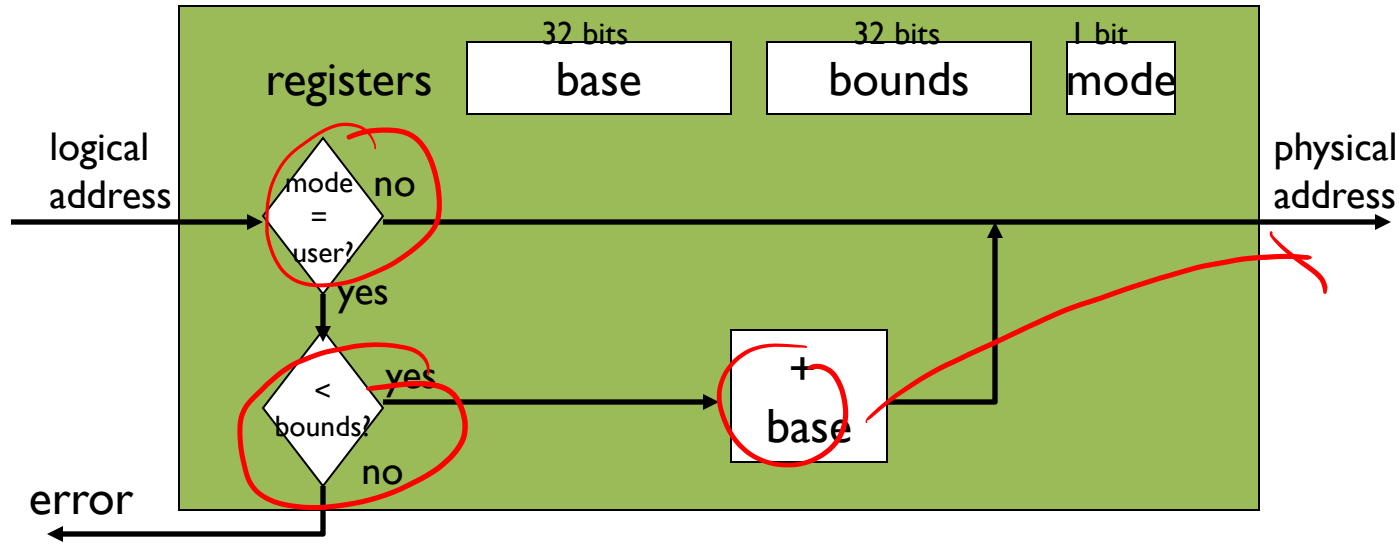
Implementation of BASE+BOUNDS

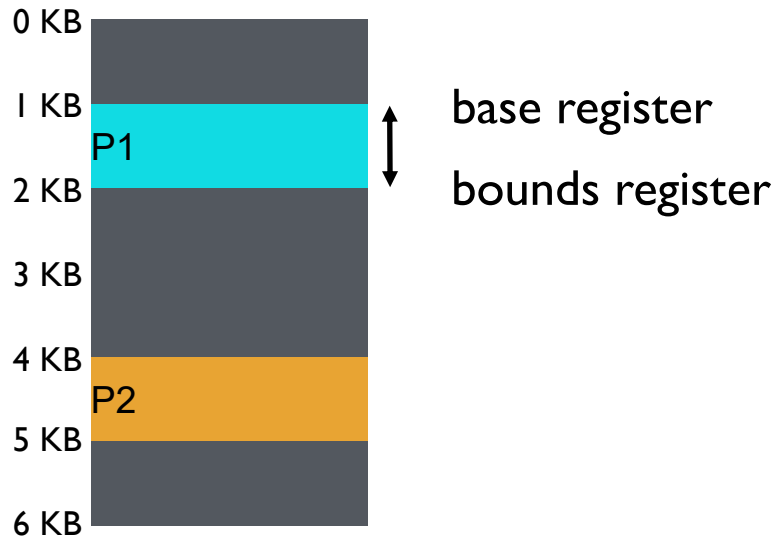
Translation on every memory access of user process

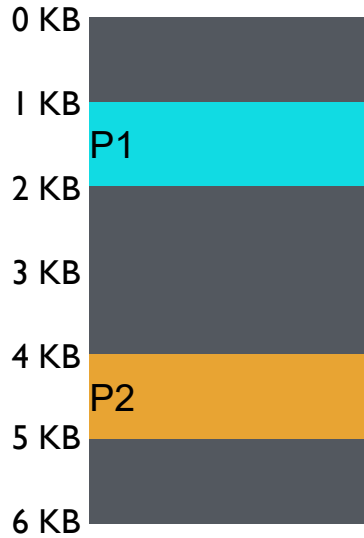
MMU compares logical address to bounds register

if logical address is greater, then generate error

MMU adds base register to logical address to form physical address







Virtual

P1: load 100, R1

P2: load 100, R1

P2: load 1000, R1

P1: load 100, R1

P1: store 3072, R1

Physical

load 1124, R1

load 4196, R1

load 5196, R1

load 2024, R1

3072 > 1024
interrupt OS!

Can P1 hurt P2?

Managing Processes with Base and Bounds

Context-switch: Add base and bounds registers to PCB

Steps

- Change to privileged mode
- ~~Save base and bounds registers of old process~~
- Load base and bounds registers of new process
- Change to user mode and jump to new process

Protection requirement

User process cannot change base and bounds registers

User process cannot change to privileged mode

Base and Bounds Advantages

Provides protection (both read and write) across address spaces

Supports dynamic relocation

- Can place process at different locations initially and also move address spaces

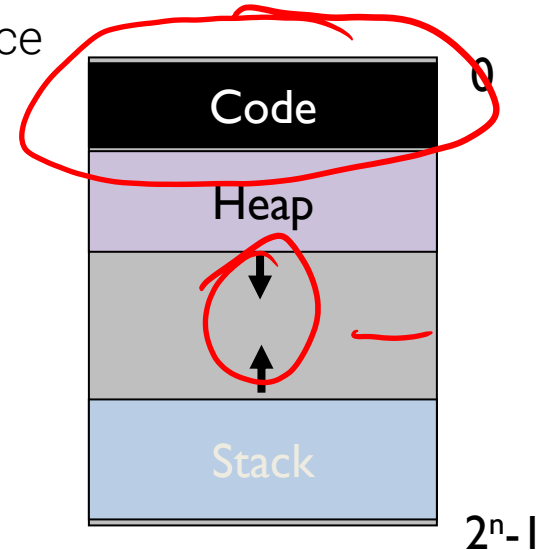
Simple, inexpensive implementation: Few registers, little logic in MMU

Fast: Add and compare in parallel

Base and Bounds Disadvantages

Disadvantages

- Each process must be allocated contiguously in physical memory
Must allocate memory that may not be used by process
- No partial sharing: Cannot share parts of address space



<https://go.wisc.edu/1zxyu9>

Quiz



Quiz – True/False

The MMU dynamically translates virtual addresses at every memory reference.

The base register contains a virtual address.

While using static relocation, a process' address space cannot be moved after it has been placed at a specific location.

QUIZ – MULTIPLE CHOICE

Where is **z** located in address space?

a.Heap

b.Stack

c.Code

d.No allocation needed

```
#include <stdio.h>
#include <sys/types.h>
```

```
int main()
{
    int *z = malloc(sizeof(int));
    return 0;
}
```

Summary

Operating systems virtualize memory to support multiprogramming

- Each process gets its own address space
- Processes are prevented from accessing each other's memory

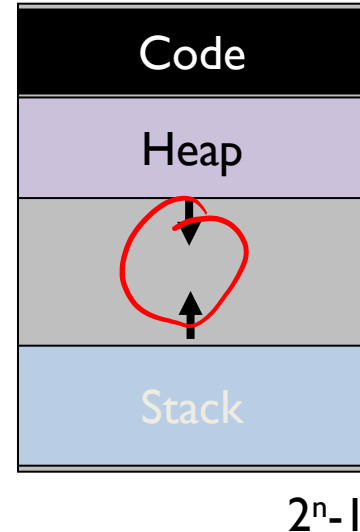
Hardware support required for efficient execution

5) Segmentation

Divide address space into logical segments

- Each segment corresponds to logical entity in address space
(code, stack, heap)

Each segment has separate base + bounds register



Segmented Addressing

Process now specifies segment and offset within segment

How does process designate a particular segment?

- Use part of logical address
 - Top bits of logical address select segment
 - Low bits of logical address select offset within segment

What if small address space, not enough bits?

- Implicitly by type of memory reference
- Special registers

Segmentation Implementation

MMU contains Segment Table (per process)

- Each segment has own base and bounds, protection bits
- Example: 14 bit logical address, 4 segments;

f = 111
G = 0110

How many bits
for segment?

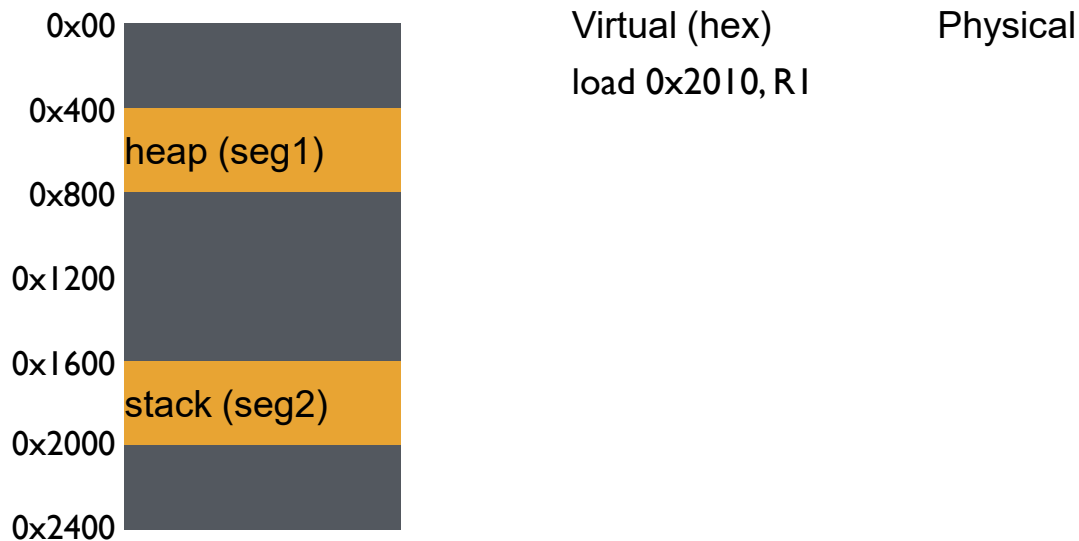
How many bits
for offset?

12

Segment	Base	Bounds	R	W
<u>0</u> 60	0x2000	0x6ff	1	0
<u>1</u> 01	0x0000	0x4ff	1	1
<u>2</u> 10	0x3000	0xfff	1	1
<u>3</u> 11	0x0000	0x000	0	0

remember:
1 hex digit → 4 bits

Visual Interpretation

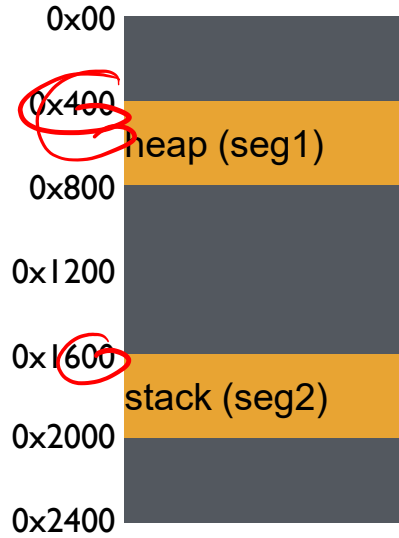


Segment numbers:

0: code+data

1: heap

2: stack



Virtual
load 0x2010, R1

load 0x1010, R1

load 0x1100, R1

Physical

$$0x1600 + 0x010 = 0x1610$$

$$0x400 + 0x010 = 0x410$$

$$0x400 + 0x100 = 0x500$$

Segment numbers:

0: code+data

1: heap

2: stack

Practice

Segment	Base	Bounds	R W
0	0x2000	0x6fff	1 0
1	0x0000	0x4fff	1 1
2	0x3000	0xffff	1 1
3	0x0000	0x0000	0 0

Remember:

1 hex digit → 4 bits

Translate logical (in hex) to physical

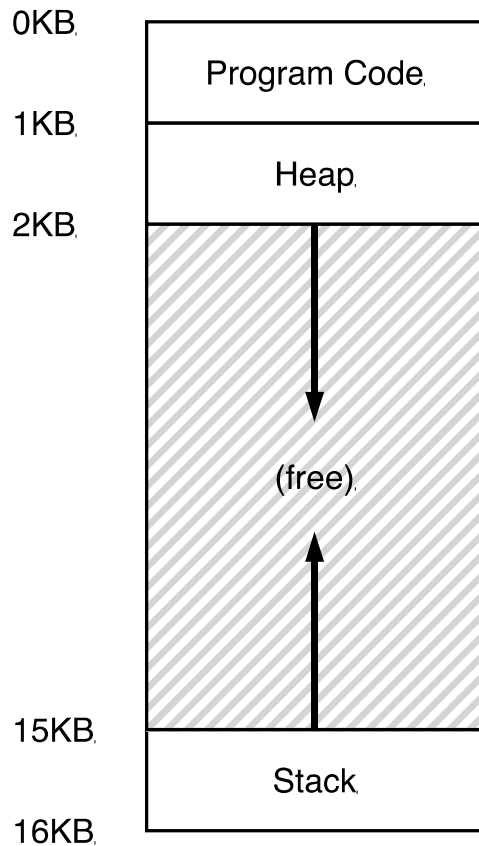
0x0240:

0x1108:

0x265c:

0x3002:

HOW DO STACKS GROW ?



Stack goes 16K $\rightarrow$ 12K, in physical memory is 28K $\rightarrow$ 24K

Segment base is at 28K; size is 4K

Virtual address 0x3C00 = 15K

$\rightarrow$ top 2 bits (0x3) segment ref, offset is 0xC00 = 3K

How do we make CPU translate that ?

Add "grows down" bit to segment!

Negative offset = subtract segment size from offset

= 3K (offset) - 4K (size) = -1K from top

Add to base = 28K - 1K = 27K; 0x3C00 $\rightarrow$ 27K phys

HOW DOES THIS LOOK IN x86

Stack Segment (SS): Pointer to the stack

Code Segment (CS): Pointer to the code

Data Segment (DS): Pointer to the data

Extra Segment (ES): Pointer to extra data

F Segment (FS): Pointer to more extra data

G Segment (GS): Pointer to still more extra data

Advantages of Segmentation

Enables sparse allocation of address space

Stack and heap can grow independently

Heap: If no data on free list, dynamic memory allocator requests more from OS (e.g., UNIX: malloc calls `sbrk()`)

Stack: OS recognizes reference outside legal segment, extends stack implicitly

Different protection for different segments

Enables sharing of selected segments

Read-only status for code

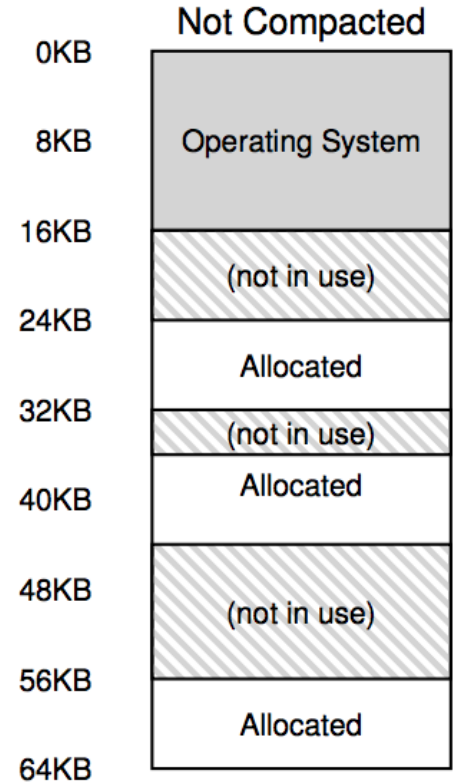
Supports dynamic relocation of each segment

Disadvantages of Segmentation

Each segment must be allocated contiguously

May not have sufficient physical memory for large segments?

External Fragmentation



Practice: Segmentation

	Segmentation	Static Relocation	Base	Base+Bounds	Time sharing
add per-process starting location to virt addr to obtain phys addr					
rewrite code and addresses before running					
several base+bound pairs per process					
one process uses RAM at a time					
dynamic approach that verifies address is in valid range					